

《新生研讨课》报告

「昨日圆车」六边形战棋的设计与实现

班 级: _____

姓 名: _____

学 号: _____

教 师: 任　　骜

重庆大学　本科生院

昨日圆车（HEX WAGON）——基于六边形网格的双人回合制战棋游戏

作者 1　　作者 2　　作者 3　　作者 4
第 XX 班　第 XX 组

摘要：本文围绕《新生研讨课》的实践要求，设计并实现了一款运行于浏览器、支持双人实时对战的六边形回合制战棋游戏「昨日圆车」。（1）目标：在尽量少的技术栈内做出一款规则完整、可复现实验、可公网联机的小体量策略游戏，并用它检验「规则层与界面层解耦」这一工程方法是否可行。(2)方法：以轴向坐标（axial coordinates）对边长为 N 的六边形地图建模；把放置合法性、射程判定、大本营结算与断联移除等规则实现为不依赖渲染的纯逻辑引擎，并采用 UMD 写法让浏览器与 Node 服务端共用同一份规则代码；界面层用 Canvas 2D 绘制；单人模式采用「评估函数 + 一步前瞻」的启发式 AI；联机模式采用 WebSocket 传输、服务端权威状态同步；此外以数据驱动的方式设计了 7 关新手教学。（3）结果：完成本地双人、单人 AI、联机对战、新手教学四种模式；规则引擎、AI 与联机协议分别通过 599 项、84 项与 38 项自动化断言；AI 对随机策略取得 19 胜 0 负；系统部署于公有云，可通过公网完成双人实时对局，并支持全部数值在设置中调整。

关键词：六边形网格；回合制策略游戏；游戏规则引擎；游戏人工智能；WebSocket；Web 前端

1　引言

1.1　研究背景

电子游戏是软件工程与人工智能的典型交叉载体。与一般的增删

改查系统相比，游戏对「状态一致性」和「规则正确性」的要求要苛刻得多：任何一个判定分支写错，都会直接表现为玩家可见的错误结果，而且很难靠人工穷举发现。在各类游戏中，回合制战棋（strategy wargame）尤其适合作为课程实践的载体——它没有实时操作的压力，规则可以完全形式化，状态空间规模适中，能够在一学期内完成从需求到部署的闭环，同时留出足够的空间去讨论平衡性与算法优化。

从学习角度看，做一个战棋游戏会同时遇到四类问题：几何建模（如何在网格上表示位置、邻接和距离）、规则工程（如何把「同时结算」这类全局判定写对且可测）、算法设计（如何在合理的计算量内做出不犯傻的决策）以及系统集成（如何让两台设备上的游戏状态保持一致）。这四个问题恰好覆盖了程序设计、数据结构、算法与计算机网络的核心内容。

1.2 本文研究的具体问题

本文要解决的核心问题是：如何用尽量少的技术栈，实现一款规则完备、可双人实时对战、并且可以随时调整数值做平衡实验的战棋游戏。围绕这个问题，拆成三个子问题：（1）网格与邻接的表示——如何用最少的状态描述一张边长可调的大六边形地图，并让「只能放在己方连通块边缘」「射程 2 能打到第 2 圈」这类规则只依赖整数运算；（2）回合制结算的一致性——伤害判定是「同时」发生的，死亡之后还要连锁处理失去补给的棋子，如何保证同一份规则在单机、AI 与联机三种场景下结果完全一致；（3）实时同步——联机时双方的判定绝不能各自为政，如何在弱网、断线与误操作下保证状态收敛。

1.3 国内外研究现状

六边形网格在战棋与策略游戏中被广泛采用，其坐标系统已有成熟总结：Red Blob Games 系统地给出了偏移坐标、立方坐标与轴向坐标三种表示及相互转换，并给出了距离、邻接环、直线与范围查询的通用算法[1]，本文的网格层基本沿用了这套方案。在博弈智能方面，

传统做法是「极小极大搜索 + 手工评估函数」，深蓝即以此在 1997 年击败国际象棋世界冠军[2]；此后蒙特卡洛树搜索（MCTS）把随机模拟引入博弈树，显著降低了对评估函数精度的依赖[3]；AlphaZero 进一步用自对弈与深度网络替代了人工特征，在国际象棋、将棋与围棋上达到超人类水平[4]。在实时 Web 方面，IETF 在 RFC 6455 中定义了 WebSocket 协议[5]，为浏览器与服务器之间的全双工通信提供了标准方案，这也是当前在线对战类网页游戏最常用的传输方式。此外，游戏 AI 的工程实践已有系统总结[6][7]，尤其是「感知—决策—行动」的分层结构，对本文的 AI 模块设计有直接参考价值。

需要说明的是，上述研究多针对算力充足、状态空间极大的场景。本文面对的是一个单局 30～60 回合、每回合仅 3 点费用的轻量战棋，其动作空间远小于国际象棋，因此并不需要 MCTS 或神经网络，一个设计良好的评估函数配合一步前瞻就足以提供可玩的对手——这一点在后文 3.3 节的实验数据中得到了验证。

1.4 本文采用的方法

本文采用「规则层与界面层解耦 + 服务端权威」的总体思路。具体包括四点：（1）把全部规则写进一个无副作用的纯逻辑引擎 engine.js，界面只负责把状态画出来、把点击翻译成动作，规则正确性由引擎的自动化测试来保证；（2）引擎采用 UMD 写法，浏览器通过 <script> 加载、Node 端通过 require() 加载，前后端共用同一份代码，从结构上消除「客户端能下、服务端不认」的分歧；（3）联机状态由服务端裁定并广播，客户端只做乐观渲染与断线重连；（4）把所有数值（地图边长、障碍密度、每回合费用、先手方、后手补偿、三个兵种的四项属性、大本营血量）抽成设置项，使平衡性实验不需要改代码。

1.5 主要结果

最终实现的系统包含四种模式：本地双人（同一设备轮流操作）、

单人 AI、联机对战与新手教学。规则引擎与联机协议分别通过 599 项与 38 项断言，AI 通过 84 项断言；AI 对随机策略的胜率为 19:0；系统以静态资源 +Node 服务端的形式部署于公有云，公网可直接进行双人实时对局；7 关新手教学覆盖了放置、费用、血量、结算、射程、断联与拆基地等全部核心规则。

2 文献综述

2.1 六边形网格与战棋规则建模

六边形网格相对正方形网格的最大优势在于「邻接距离更均匀」：正方形四邻域的欧氏距离并不相等，斜向移动会更快，而六边形每个格子到六个邻居的距离完全一致，这使得射程、移动力这类以「步数」计量的规则更自然。Red Blob Games 的综述指出，六边形网格常用的表示有三种：偏移坐标（offset）、立方坐标（cube）与轴向坐标（axial）[1]。其中立方坐标用三个整数 (x, y, z) 且恒有 $x + y + z = 0$，几何意义最直观，距离可以用 $\max(|\Delta x|, |\Delta y|, |\Delta z|)$ 直接计算；轴向坐标相当于把立方坐标中的一个分量省掉，只用 (q, r) 两个整数即可唯一确定格子，是存储效率与计算便利性的折中。本文选择了轴向坐标，原因是它既避免了偏移坐标在奇偶行之间的分支判断，又比立方坐标少存一个冗余分量。

在规则建模方面，教科书式的做法是把「地图、棋子、规则」三者分离：地图只描述哪些格子可通行，棋子只描述属性与归属，规则则以查询的方式在地图与棋子上做判定[6]。这种分离的最大好处是规则可以被单独测试。以本文的「放置合法性」为例，规则要求棋子只能放在「与己方大本营连通的那片棋子」的边缘，这本质上是一次图的广度优先搜索；而「射程判定」则退化为两次坐标差的整数运算。二者都不涉及像素坐标，因此可以在没有渲染环境的命令行里被完整验证。Red Blob Games 还特别讨论了地图对称性的问题：如果障碍物是随机生成的，就必须保证双方地形等价，否则先手优势会被地形

放大。这一点直接影响了本文的设计——地图障碍采用中心对称成对生成[1]。

2.2 博弈树搜索与评估函数

棋类 AI 的经典框架是极小极大搜索配合局面评估函数。深蓝的成功说明了两个要素的重要性：足够深的搜索与高质量的评估函数[2]。当状态空间继续增大时，穷举搜索不再可行，蒙特卡洛树搜索通过随机模拟与置信上界选择来平衡「探索」与「利用」，在围棋等评估困难的博弈上取得了突破[3]；Browne 等人的综述系统总结了 MCTS 的四个阶段（选择、扩展、模拟、回溯）及其各种变体[3]。AlphaZero 则进一步把策略网络与价值网络引入 MCTS，用自对弈替代人类棋谱，在国际象棋、将棋与围棋上均超过当时的专用程序[4]。

对于本文这样的小体量战棋，直接套用 MCTS 并不划算：单回合的可行动作数在几十到上百量级（每个合法落点、三种兵种、是否结算的任意组合），模拟一次完整对局需要数百次状态转移，在浏览器主线程上会给交互带来明显卡顿。因此本文选择了更「轻」的方案：以评估函数为核心，配合一步前瞻。评估函数把局面折算成一个标量，包含棋子价值、受伤折价、推进程度与施加压力四项；一步前瞻则对「放哪个子、是否结算」的候选动作逐个模拟，取评估值最优者。Buckland 在《Programming Game AI by Example》中把这种结构称为「感知—决策—行动」的简化版，并指出在动作空间有限时，评估函数的工程质量比搜索深度更影响最终表现[6]，本文的实验结果与之吻合。

2.3 实时联机与浏览器渲染

网页端的实时对战有两条传统路线：轮询（polling）与长连接。前者实现简单但延迟高、开销大；后者以 WebSocket 为代表，通过一次 HTTP 升级握手建立全双工通道，之后双方可直接收发消息帧[5]。RFC 6455 定义了握手流程、帧结构与关闭语义，并在协议层内

置了 ping/pong 心跳机制，这正是本文联机模块的基础。在架构上，实时对战通常采用「服务端权威」（server-authoritative）模型：客户端只发送意图，服务端校验并广播结果，从而避免客户端篡改与两端状态分歧。Yannakakis 与 Togelius 在《Artificial Intelligence and Games》中把这种结构归纳为「游戏状态由权威方持有、玩家输入作为事件流」的模式，并指出它对反作弊与回放的支持最好[7]。

渲染方面，浏览器提供了 DOM、SVG 与 Canvas 三种绘制手段。对于本文这种「几十个六边形 + 若干棋子 + 动画特效」的场景，Canvas 2D 的优势在于一次绘制调用即可完成整帧重绘，且易于实现虚线高亮、渐变与飘字等效果；代价是需要自己处理设备像素比（devicePixelRatio）、窗口缩放与命中检测。实践中我们遇到一个典型问题：在 CSS 弹性布局下若直接按容器尺寸设置 canvas 的 width/height 属性，会触发「设置尺寸 → 容器变化 → 再次 resize」的反馈循环，最终表现为画布不断放大；解决办法是在读取尺寸之前先把画布清零。这类工程细节通常不会出现在论文中，但对可用性影响很大，故在 3.2 节一并记录。

3 研究内容

3.1 系统架构设计

系统采用四层结构，自上而下分别为表示层、应用层、规则层与传输层，如图 1 所示。层与层之间是单向依赖：上层可以调用下层的接口，下层不感知上层的存在。这样做的直接收益是规则层可以在完全脱离浏览器的环境（Node.js）中被测试，也能被服务端直接复用。



图 1　系统分层架构与同构复用关系

表示层由两个页面组成：开始菜单页（index.html + menu.js）负责模式选择、参数设置与地图预览；对局页（game.html + main.js）负责把引擎状态画出来，并把点击事件翻译成动作。绘制逻辑集中在 renderer.js 中，它只接收「当前应当显示什么」的描述对象，不持有任何游戏状态。应用层包含三个相对独立的模块：ai.js 负责决策，net.js 负责与服务端通信，tutorial.js 以纯数据的形式描述教学关卡。规则层是系统的核心，hex.js 负责几何运算，engine.js 负责状态与回合，maps.js 与 settings.js 负责地图与参数。传输层的 server.js 同时承担静态托管与 WebSocket 房间管理。

这种分层带来两个具体好处。第一，规则的可测性：由于 engine.js 不触碰 DOM，测试脚本可以直接在 Node 中构造局面、执行动作并断言结果，本文的 599 项规则断言全部以这种方式运行，执行时间不到 1 秒。第二，联机的一致性：服务端 require 的是同一个 engine.js，因此「客户端认为合法」与「服务端认为合法」在代码层面就是同一个判断函数，不存在两份规则逐渐漂移的风险。

3.2　技术方案

3.2.1 六边形地图与坐标系统

地图是一张由小六边形拼成的大六边形，边长 N 可调（3～8）。格子用轴向坐标 (q, r) 表示，其像素位置由 x = size·1.5·q、y = size·√3·(r + q/2) 唯一确定，无需为每个格子缓存坐标。地图范围由三个不等式共同给出：|q| ≤ N−1、|r| ≤ N−1、|q+r| ≤ N−1，格数恰为 3N(N−1)+1，N=5 时为 61 格。六个邻居的方向向量固定为 (±1,0)、(0,±1)、(+1,−1)、(−1,+1)，两点距离用立方坐标范数 d = (|Δq| + |Δr| + |Δq+Δr|)/2 计算，全部是整数运算，如图 2 所示。

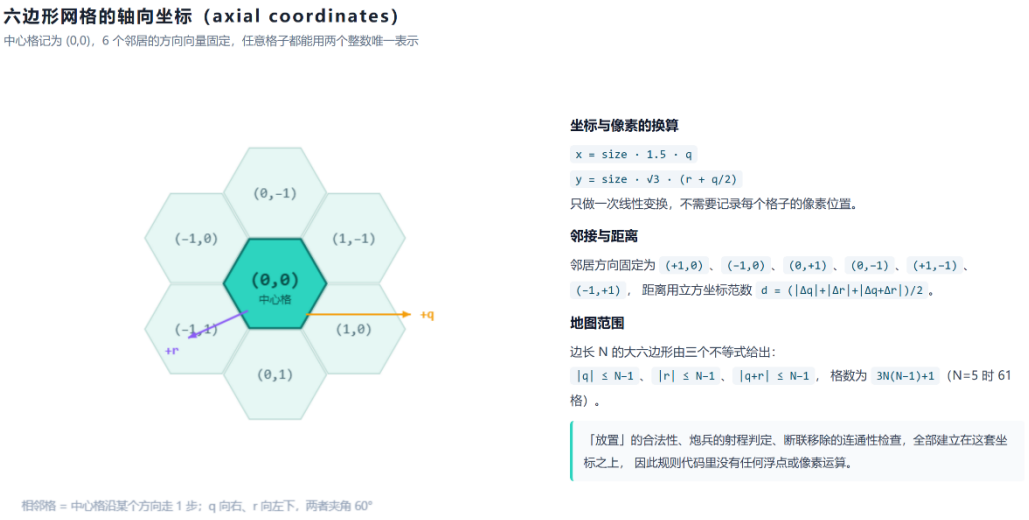


图 2 轴向坐标、邻居方向与棋盘范围

两个大本营固定在地图的上下顶点，坐标分别为 (0, −(N−1)) 与 (0, N−1)。障碍物的生成采用中心对称：先在候选格中随机抽取一个格子放置障碍，同时把它的中心对称点 (−q, −r) 一起标为障碍，从而保证双方地形严格等价、且两端大本营互为镜像。生成过程还包含两道校验：大本营及其邻格不允许出现障碍；生成后对双方的连通块做一次广度优先搜索，若某一方无法从大本营出发建立连通块则整张图重新生成。此外，本文还提供了 8 张固定种子的「默认地图」，只存「密度 + 种子」两个数字，由同一套生成器确定性地产出，因此同一张图在任何设备上完全一致。

3.2.2 规则引擎

引擎的核心状态包括：地图格子、棋子集合（以格子为键的映射）、当前行动方、当前回合数、当前费用、双方的大本营血量以及击杀与损失统计。一次完整的「放置—结算—结束回合」流程如下。

放置的合法性判断依次为：对局未结束、轮到该方、费用足够、目标格在地图内、不是障碍、没有棋子、且属于「己方连通块的边缘」。最后一条的实现是对该方全部棋子做一次广度优先搜索，把与自身大本营连通的集合求出，再取其所有邻居格作为合法落点集合。开局时双方各只有大本营，因此第一回合恰好只有 3 个合法落点。

结算是本系统最需要小心的一步。它分为三个阶段：第一阶段计算「伤害图」，对场上每一个棋子（含大本营），遍历所有敌方棋子，若二者距离不大于攻击方的射程，则把攻击方的攻击力累加到该棋子身上；第二阶段按「血量 ≤ 攻击之和」同时判定死亡，由于用的是结算前的局面，双方完全可能在同一轮同时损失棋子；第三阶段处理补给线，把死亡棋子移除后，对双方重新做连通性搜索，与己方大本营失去连接的棋子一并移除。三个阶段都必须基于同一份快照，否则会出现「先死的一方少挨一刀」这类顺序依赖。结算不消耗费用、不结束回合，回合结束前可以反复执行，这也是本游戏的核心策略点之一。

为了让联机与回放成为可能，引擎实现了完整的序列化：toJSON 输出纯数据结构，fromJSON 逐字段重建，clone 则用于 AI 的候选动作模拟。所有状态（包括击杀统计、单次结算的最大击杀数）都会被序列化，因此服务端广播的状态与客户端本地的状态在结构上完全一致。

3.2.3　渲染与交互

渲染层使用 Canvas 2D，每一帧按「地板与障碍—落点高亮—棋子—血条—飘字特效」的顺序绘制。为了保证高分屏不模糊，画布的实际像素尺寸按 devicePixelRatio 放大，再用样式尺寸缩回逻辑尺寸；窗口尺寸变化时先清空画布再测量容器，避免「测量—设置—再测量」

的反馈循环。落点高亮使用虚线描边并在费用不足时改为红色，棋子被结算击杀时在格子上飘出「已造成/需要」的数字（例如 3/4 表示已吃 3 点、离致死还差 1 点），这一细节显著降低了新手理解结算规则的成本。

交互层把鼠标与触摸事件统一为「点击某格」的语义，再交给动作分发函数。为了让规则在三种模式下保持一致，界面层不直接修改任何状态，而是通过引擎的公共方法（place / resolve / endTurn）推进，并在每次推进后重新渲染。键盘快捷键（1/2/3 选兵种、R 结算、空格结束回合）与按钮共用同一套分发逻辑。

3.2.4 人工智能设计

AI 采用「评估函数 + 一步前瞻」。评估函数把局面折算为标量，包含四项：棋子价值（按兵种与剩余血量折算，受伤棋子按比例折价）、推进程度（己方棋子向对方大本营靠近的程度）、压迫程度（对对方大本营的潜在伤害）以及机动性（合法落点的数量）。在此基础上，AI 的每回合决策分两步：先枚举所有「兵种 × 合法落点」的放置动作，对每个候选动作克隆局面并模拟执行，取评估值最优者；随后再用同样的方式判断「现在是否值得结算一次」，只有当结算能改善局面时才执行。为了不阻塞界面，AI 的每一步之间插入约 300 毫秒的延时，玩家可以看到对手一步步摆子的过程。

此外，AI 还被用于平衡性实验：让两个参数相同的 AI 互相对弈并统计回合数，可以粗略反映不同地图尺寸下对局的收敛速度。实测表明边长 4 的棋盘平均约 17 回合结束，边长 3 约 12 回合，而边长 6 以上经常出现长时间僵持，这为默认值选择边长 5 提供了依据。

3.2.5 联机与状态一致性

联机采用房间制：创建者获得一个 4 位数字房号与一枚座位令牌，加入者凭房号进入另一个座位，两人同时在线即自动开局。传输层使用 WebSocket，服务端每 25 秒发起一次 ping 以穿透中间设备；

客户端每 45 秒发送一次应用层心跳，既用于保活，也让免费云托管平台判定实例处于活跃状态而不休眠。断线后客户端凭令牌自动重连并回到原座位，服务端为每个座位保留 10 分钟的重连窗口；若对局已结束且双方都离开，房间立即回收。

所有游戏动作（放置、结算、结束回合、认输、求和）都先发送到服务端，服务端用同一份 engine.js 校验后执行，再把新状态与事件广播给双方。客户端收到状态后整体替换本地状态而不是增量修改，从根上避免了「增量丢失导致两端分叉」。非法动作（例如不是自己回合、费用不足）会被服务端拒绝并回传错误提示。求和是唯一允许在非自己回合应答的动作，因此被单独提前处理。

3.2.6 新手教学与可配置性

新手教学由 7 个预设小局面组成，每关只教一个知识点：放置棋子、费用限制、坦克的血量优势、结算与伤害、炮兵的射程、断联移除、拆掉大本营。每关以纯数据描述：地图尺寸、初始费用、需要预置的棋子（通过引擎的 spawn 接口无视放置规则直接落子）、可用兵种，以及若干「步骤」，每个步骤包含一句提示和一个判定函数。控制器在每次动作后依次检查当前步骤是否完成，完成后自动推进；若玩家做出了与本关目标无关的操作（例如把坦克放错位置），关卡会自动重置并给出提示。这种「数据 + 判定函数」的写法让关卡可以在 Node 中被完整测试，也便于后续继续扩充。

可配置性是本项目的另一条主线。地图来源（随机 /8 张默认地图）、边长、障碍密度、每回合费用、先手方、后手补偿、三个兵种的属性、大本营血量都可以在设置中修改，并保存在浏览器本地。设置一旦改变，菜单摘要会即时更新；联机时这些参数由建房的玩家一次性下发给服务端，服务端据此创建对局，双方看到的地图与规则完全一致。

3.3 项目结果与展示

系统最终拆分为两个页面：开始菜单页（图 3）与对局页（图 4）。

菜单页提供四种模式的入口、房号快速加入、服务器状态与当前房间数；对局页左侧为双方状态面板与战报，中间为棋盘，右侧为速查与教学面板，底部为兵种商店与操作按钮。



图 3　开始菜单页（四种模式、快速加入、服务器状态）

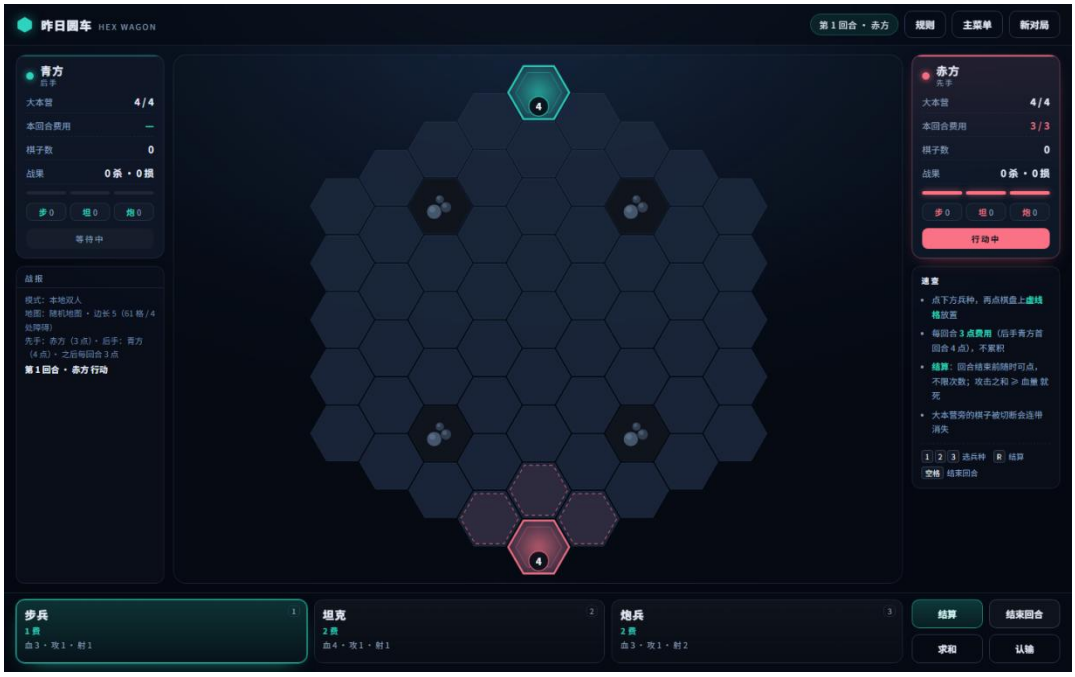


图 4　对局页（棋盘、双方状态、战报与兵种商店）

新手教学的第一关与第三关分别如图 5 所示。教学中会屏蔽与

当前关卡无关的兵种，并在底部只保留必要的操作按钮，玩家点「结算」后可以立即看到伤害数字飘出，从而把抽象规则与可视化结果对应起来。



图 5　新手教学（左：放置棋子；右：坦克扛伤害）

对局结束时给出本局的对比战绩（图 6），包括双方击杀、损失、剩余棋子与单次结算的最大击杀数，并统计总回合数、耗时与地图规模。这些数据既是对局复盘材料，也为后续的平衡性调整提供了依据。



图 6　结算界面与战绩对比

系统的功能与自动化测试情况如表 1 所示。除功能测试外，本文还做了一组简单的 AI 对抗实验：让 AI 与随机策略对弈 19 局，AI 全部获胜；让两个 AI 互相博弈，边长 4 时平均约 17 回合结束，边长 3 时约 12 回合，边长 6 以上经常出现长时间僵持。这些数据说明评估函数确实能区分「好棋」与「坏棋」，同时也暴露出在更大棋盘上缺乏长远规划能力的不足。

功能模块	实现情况	验证方式 / 结果
六边形地图（边长 3~8）	轴向坐标、中心对称障碍、8 张固定默认图	边界与对称性断言全部通过
三种兵种与费用	步兵 1 费、坦克 2 费、炮兵 2 费，属性可调	599 项规则断言通过
放置合法性	连通块边缘 + 障碍/占位/费用判定	含边界与非法落点用例
结算与断联移除	同时判定、三阶段处理、可重复结算	含同时死亡与连锁移除用例
胜负与大本营	攻击合计 ≥ 血量即摧毁，	含认输、求和与同时拆家

功能模块	实现情况	验证方式 / 结果
	平局可判定	
单人 AI	评估函数 + 一步前瞻	对随机 19:0；84 项断言通过
联机对战	WebSocket + 服务端权威 + 断线重连	38 项协议断言通过
新手教学	7 关数据驱动关卡	每关目标与预置局面均有断言
部署	静态资源 + Node 服务端 + 进程守护	公网双人实时对局可用

4 结论

本文完成了「昨日圆车」这款六边形双人回合制战棋游戏的设计与实现，主要工作可以归纳为三点。第一，在几何层面选用轴向坐标描述六边形网格，把放置合法性、射程判定与补给连通性全部归结为整数运算与图搜索，使规则代码不依赖任何渲染环境。第二，在工程层面采用「规则层与界面层解耦 + UMD 同构复用」的方案，让浏览器与服务端共享同一份规则代码，从结构上消除了联机场景下的判定分歧，并使规则可以用 599 项断言在 1 秒内完成回归。第三，在玩法层面把全部数值抽成可配置项，并以数据驱动的形式实现了 7 关新手教学，使系统既能用于对局，也能作为平衡性实验与规则教学的平台。

本系统仍有明显的不足。其一，AI 只做了一步前瞻，缺乏跨回合的规划，在开阔地图上容易被针对；受此影响，边长 6 以上的对局经常难以收敛。其二，平衡性数据还不够充分，目前只做了 AI 互搏与 AI 对随机的粗略实验，没有引入人类对局的统计，结论的置信度有限。其三，功能上尚缺观战、回放与匹配机制，联机依赖房号手工传递，规模化体验不足。

后续可以从三个方向改进：一是把搜索从一步前瞻扩展到带剪枝的深度搜索或轻量 MCTS，并在评估函数中引入阵型与威胁评估；二是引入基于 Elo 的 AI 参数自校准与对局数据回收，用真实数据替代经验设定默认值；三是补齐观战与回放（借助已有的序列化能力，回放只需存储初始局面与动作序列），并尝试用云端房间列表替代手工房号。

参考文献

[1] Patel A. Hexagonal Grids[EB/OL]. Red Blob Games, 2013[2026-11-01]. https://www.redblobgames.com/grids/hexagons/.

[2] Campbell M, Hoane A J, Hsu F H. Deep Blue[J]. Artificial Intelligence, 2002, 134(1/2): 57-83.

[3] Browne C B, Powley E, Whitehouse D, et al. A survey of Monte Carlo tree search methods[J]. IEEE Transactions on Computational Intelligence and AI in Games, 2012, 4(1): 1-43.

[4] Silver D, Hubert T, Schrittwieser J, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play[J]. Science, 2018, 362(6419): 1140-1144.

[5] Fette I, Melnikov A. The WebSocket Protocol: RFC 6455[S]. Internet Engineering Task Force, 2011.

[6] Buckland M. Programming Game AI by Example[M]. Plano: Wordware Publishing, 2005.

[7] Yannakakis G N, Togelius J. Artificial Intelligence and Games[M]. Cham: Springer, 2018.